Architecture Deep-Dive

Interview Preparation Guide

Target Audience:

Depth:

Topics:

Stack:

Searchly Platform · June 2026

Table of Contents

SECTION 0

Production Scale Sheet

Real numbers from a live deployment — single-node CPU-only VM (June 2026)

These are REAL numbers pulled from a live production deployment. Use them to answer 'How many users?', 'What traffic?', 'What latency?' in interviews.

Corpus — What Is Indexed

Metric	Count	Notes
Total documents in OpenSearch	421,117	Documents index (BM25 only)
→ Jira issues	232,509	55% of corpus — bugs, sprints, field data
→ Confluence pages	150,992	36% — runbooks, architecture, procedures
→ Git files	37,616	9% — deployment YAML, code, markdown
Total chunks in OpenSearch	981,681	Chunks index — BM25 + 384-dim HNSW vectors
Avg chunks per document	2.33	2000-char chunks, 200-char overlap
Distinct Git repositories	7	Deployment repos: YAML-heavy (33K YAML files)
Knowledge graph entities	40,264	In Postgres kg_entities; relationships = 0 (not wired)
Git languages indexed	YAML 33K, Markdown 1.8K, Python 1.6K, JSON 682	Deployment-config heavy — mirrors real customer infra repos

Tenants & Customers

Metric	Count	Notes
Formal tenants (Postgres)	5	Each tenant gets isolated Kafka topic (ENTERPRISE tier)
Customers in registry	57	Auto-registered from DevOps repo branch parsing
→ Production lifecycle stage	46	Have live k8s cluster configured
→ Testing lifecycle stage	9	Cluster not yet production

Metric	Count	Notes
→ Solution design stage	2	No cluster; knowledge-only (RAG) answers only
Users	14	Across all tenants

Storage — Disk & RAM

Component	Disk / RAM	% Used	Notes
Documents index (BM25, no vectors)	848 MB	—	3 shards, 0 replicas (dev). Keyword + text fields.
Chunks index (BM25 + HNSW 384-dim)	10.1 GB	—	3 shards, 0 replicas. knn_vector M=16 ef=128 Lucene.
Total OpenSearch storage	10.9 GB	—	Single node. No replication overhead.
OpenSearch JVM heap	310 MB used / 512 MB	60%	Running on constrained dev limits.
Total VM disk used	147 GB / 194 GB	76%	Includes Docker images, volumes, OS.

At this scale: **~421K docs / 982K chunks** on a single-node OpenSearch with CPU-only inference. Production target is K8s with dedicated node pools and GPU for embedding.

Service Memory Footprint (Live docker stats, idle)

Service	RAM Used	Limit	% of Limit	Notes
OpenSearch	1.5 GB	2 GB	75%	Heap 310 MB + HNSW segments off-heap
Embedding service (BGE models)	1.37 GB	2 GB	68%	bge-small-en-v1.5 + bge-reranker-base both loaded
Ollama (llama3.2:3b)	994 MB	5 GB	20%	Model pre-loaded in RAM; cold load ~8s
Kafka	715 MB	1 GB	70%	12-partition enterprise topic + consumer offsets
search-api (Spring Boot)	669 MB	2 GB	33%	Includes Spring context, Caffeine caches, Hikari pool
Postgres	292 MB	512 MB	57%	Documents, tenants, KG tables

Service	RAM Used	Limit	% of Limit	Notes
Gateway (Spring Cloud)	213 MB	512 MB	42%	JWT validation, rate limiter, routing
Intelligence agent (FastAPI)	59 MB	1 GB	6%	Async Python — very lean
Indexer (Spring Boot)	32 MB	256 MB	13%	Kafka consumer + bulk OpenSearch writer
Redis	8.7 MB	256 MB	3%	60s query result cache + rate limit counters
Sync cron (Python)	46 MB	—	—	Jira/Confluence/GitHub connectors
Total Searchly footprint	**~6.0 GB**	**~14.5 GB alloc**	~41%	Can run on 16 GB VM; 32 GB recommended for headroom

Query Latency (CPU-only, no GPU, single node)

Stage	Latency	Notes
OpenSearch hybrid BM25+kNN query	~14 ms	Measured: 982K chunks, 384-dim HNSW, 6 legs × top-50
Redis cache hit (warm)	<5 ms	Measured: 60s TTL, sha256 key, sliding window invalidation
Embedding (BGE bge-small-en-v1.5)	~25 ms	Per embed call. Query gets prefix; passages do not.
Cross-encoder rerank (top-30)	~300 ms	bge-reranker-base on 30 (query, passage) pairs
Query rewrite (Ollama llama3.2:3b)	~600 ms	llama3.2:3b — small model, fast, but non-deterministic
LLM generate (Ollama llama3.2:3b)	~4,000 ms	~6 chunks as context, ~200 token answer
End-to-end full RAG (no cache)	**~5.1 s**	Retrieval legs still sequential — 250ms free with parallel futures
hits_only (no LLM, no cache)	**~1.1 s**	Rewrite + embed + retrieve + rerank. No generation.
Cache hit	**<5 ms**	Redis. Conversational queries (session_id set) never cached.

Bottlenecks in order: **LLM generate (4s)** → **query rewrite (600ms)** → **reranker (300ms)** → **retrieval legs (84ms sequential)**. Parallelising the 6 retrieval legs with CompletableFuture.allOf() saves ~250ms for free.

Connector Sync Performance

Metric	Value	Notes
Sync cycle (Track B)	Every ~4.5 hours	Jira + Confluence + GitHub; scheduler in sync-cron container
Full Jira sync (133 projects)	36,799 docs, ~18 min	Force-reindex of all issues; 283 failed (network timeouts on remote links)
KG entity extraction (Jira)	37,076 entities, ~same run	Issues → kg_entities; 0 relationships (remote link wiring incomplete)
Jira sync throughput	~34 docs/s	3 workers × 7 req/s rate limit (Atlassian API cap)
GitHub repos indexed	7 repos, 37,616 files	Streaming tarball decompression — no disk writes
GitHub language breakdown	YAML 87%, Markdown 5%, Python 4%	Deployment-config repos dominate; no git clone
Indexing throughput (Kafka→OS)	Up to 500 docs/bulk	Batch embed 50 chunks/request; bulk write to OpenSearch

Infrastructure Snapshot (Interview: 'What does it cost to run?')

Current deployment is a **single VM, CPU-only, ~37 GB RAM**. All AI inference (embedding, reranker, LLM) is on CPU. This is a staging/integration environment, not production-at-scale.

Layer	Current (staging)	Production target
Compute	1 VM, 37 GB RAM, shared CPU	K8s node pool; 3+ OS nodes, GPU node for embedding
OpenSearch	Single node, 2 GB heap, 0 replicas	3-node cluster, 8 GB heap, 1 replica
Kafka	Single broker, 12 partitions	3-broker cluster; topic replication factor 2
Embedding / Reranker	CPU (BGE small, ~25ms)	GPU T4/A10: ~3ms embed, ~30ms rerank on full 100 passages
LLM	Ollama llama3.2:3b CPU, ~4s	Ollama llama3.2:3b on GPU: ~400ms; or API-hosted model
Index scale target	421K docs / 982K chunks / 10.9 GB	5M docs / ~12M chunks / ~130 GB (see index size math)
Expected RAG latency (GPU, parallel)	5.1s (CPU sequential)	~1.2s (GPU embed+rerank, parallel retrieval legs)

SECTION 1

System Overview

What Searchly is and the business problem it solves (L1)

Searchly is a multi-tenant enterprise operational intelligence platform. It combines **hybrid BM25 + semantic (kNN) search**, a **RAG (Retrieval-Augmented Generation) pipeline**, and an **agentic intelligence assistant** that queries live Kubernetes clusters to answer real-time operational questions.

The Business Problem

Enterprise operators managing 50–200 customer deployments face three pain points:

• **Knowledge fragmentation:** Jira tickets, Confluence docs, GitHub code, and live logs exist in silos. Engineers spend hours correlating evidence manually.

• **Operational triage at scale:** When 'customer-a's core service crashes at 2am', the on-call engineer needs to know: which version is running, what the logs say, and whether there is a known Jira bug — in under 2 minutes.

• **Customer lifecycle questions:** Pre-sales engineers ask 'how would customer-b's throughput be affected with this configuration change?' — needing architecture docs + Jira history.

The One-Sentence Pitch (L1 answer)

Searchly turns your fragmented knowledge base (Jira + Confluence + GitHub + live k8s logs) into a single conversational interface — answering 'why is X broken for customer Y?' with actual log evidence and matching Jira bugs, in under 10 seconds.

System Boundaries

Boundary	What Crosses It	Protocol
User → Platform	Natural language queries via web UI or API	HTTPS / JWT
Platform → Jira/Confluence	Sync: fetch issues, pages, attachments	REST (OAuth/PAT)
Platform → GitHub	Sync: fetch repo tarballs, PR metadata	REST (PAT)
Platform → k8s cluster	Live: pod logs, deployment state, secrets	kubectl exec / k8s API
Platform → Ollama	LLM inference: rewrite, generate, rerank	HTTP (local, no egress)

SECTION 2

Service Architecture

Seven services, their roles, communication, and why they are separated (L2)

Searchly follows a **polyglot microservices pattern** — each service is sized to its I/O profile. Java handles high-throughput synchronous API paths; Python handles ML inference and async data sync; Spring Cloud Gateway handles cross-cutting concerns at the edge.

Service	Stack	Port	Role & Key Design Decision
gateway	Spring Cloud Gateway	8080	TLS termination, JWT auth, per-tenant rate limiting (Redis sliding window). The only public ingress — all internal services are not exposed.
search-api	Java/Spring Boot 3	8081	Hybrid search + document CRUD + KG API. Owns the RRF fusion logic, reranking orchestration, and RAG pipeline. Heavy CPU/IO — JVM gives GC tuning levers.
indexer	Java/Spring Boot	8082	Kafka consumer writing to OpenSearch. Stateless; scale horizontally per partition. Java chosen for OpenSearch high-level client maturity.
embedding-service	FastAPI	8083	BGE bge-small-en-v1.5 (384-dim) + cross-encoder reranker. Python because PyTorch/HuggingFace ecosystem. CPU inference — no GPU required.
intelligence-agent	FastAPI	8084	Agentic loop: Planner → Execution → Synthesis. Python for async httpx, asyncio.gather parallel tools, streaming SSE.
connectors	Python	—	Cron sync: Jira, Confluence, GitHub. Scheduled, not latency-sensitive. Python for requests library and tarfile streaming.
py-indexer	Python	—	Alternate Kafka consumer. Backup path. Useful for rapid iteration on indexing logic.

Why Microservices?

The key split is **embedding-service as a sidecar**. Embedding is compute-heavy (model load, inference) and needs independent scaling. If it were inside search-api, a JVM restart would also reload the 22MB PyTorch model. Separating it means embedding can be restarted, upgraded (different model), or horizontally scaled without touching the search path.

Service Communication Patterns

Caller	Callee	Pattern	Why Synchronous vs Async
search-api	embedding-service	Sync HTTP (Resilience4j)	Embedding is on the critical read path — must complete before RRF. Circuit breaker protects against model restart.
search-api	intelligence-agent	Sync HTTP (120s timeout)	Agent response is the final answer — must block. Long timeout because Ollama can take 4-8s.
search-api	OpenSearch	Sync HTTP (Java client)	kNN + BM25 queries: 50-100ms. Must complete for RRF merge.
search-api	Kafka	Async publish (fire-and-forget)	Index jobs are background work — 202 Accepted immediately, index later.
indexer	Kafka	Consumer group poll	Stateless consumer. Offset commit after successful OpenSearch write.
connectors	Jira/GitHub APIs	Sync HTTP (with retry)	External rate-limited APIs. Sequential with 7 req/s limit.
intelligence-agent	search-api	Async httpx (hits_only)	search_knowledge call — avoids Ollama double-call via rag_only=true flag.

KEY DESIGN: The gateway is the only service with a public TLS certificate. All internal service-to-service calls go over plain HTTP on the Docker network — simplifies cert management in dev/staging. In production (K8s), mTLS via a service mesh replaces this.

SECTION 3

Search & RAG Pipeline — Full Internals

Every step from user query to LLM answer, with latencies, algorithms, and design rationale (L3)

3.1 Complete Query Lifecycle

The pipeline has 9 stages. Understanding each stage and why it exists is the core interview topic.

Stage	Operation	Latency	Owner
0	Metadata extraction — regex pulls env (prod/staging/dev), service name, source_type from query text. Explicit params override. No LLM call.	~0ms	QueryMetadataExtractor.java
1	Redis cache check — key = sha256(tenantId + roles + query + params). HIT = immediate return. Conversational and cursor queries bypass cache.	~1ms	CacheService.java
2	Query rewrite — Ollama llama3.2:3b expands abbreviations, adds synonyms. Skipped for bare Jira keys (AES-891) — exact-ID lookups need no expansion.	~600ms	RagService.rewriteQuery()
3	Dual embed — BGE bge-small-en-v1.5 embeds both original and rewritten query. Query prefix: 'Represent this sentence: '. Runs concurrently with step 2.	~25ms x2	EmbeddingClient.java
4	6 retrieval legs — all launched concurrently via CompletableFuture.allOf() on a virtual-thread pool. Top-50 candidates each. See 3.3.	~50ms wall-clock	RagService.java
5	RRF merge — Reciprocal Rank Fusion across all 6 lists with list-weight and source-authority factors. See 3.4.	~1ms	RagService.rrfMerge()
6	Cross-encoder rerank — top-30 RRF candidates re-scored by bge-reranker-base (query, passage) cross-attention model. See 3.5.	~300ms	RerankClient.java
7	Source budget selection — greedily select top-6 while respecting per-source caps. See 3.6.	~0ms	RagService.rerank()
8	LLM generation — top-6 chunks as context, Ollama llama3.2:3b generates answer with citation instructions.	~4s	OllamaClient.java

3.2 BM25 Internals

BM25 is OpenSearch's default ranking function with **k1=1.2, b=0.75** (Lucene defaults). The formula is:

```
BM25(q,d) = SUM over terms t: IDF(t) * (tf(t,d) * (k1+1)) / (tf(t,d) + k1*(1-b + b*|d|/avgdl
))
```

Where: **IDF** = log((N - df + 0.5) / (df + 0.5)), **tf** = term frequency in document, **|d|** = doc length, **avgdl** = average doc length.

The **k1 parameter** (1.2) controls term-frequency saturation — diminishing returns on repeated terms. At k1=1.2 a term appearing 10x is only ~2x as important as appearing once. Lower k1 = faster saturation (good for short queries). The **b parameter** (0.75) controls length normalization — docs shorter than avgdl get a boost.

Recency boost is applied to the documents index (not chunks) via Gaussian decay on created_at:

```
score_final = BM25(q,d) * gauss_decay(created_at, origin=now_ms, scale=30d_in_ms, decay=0.5)
```

CRITICAL IMPLEMENTATION NOTE: created_at is mapped as long (epoch millis) NOT as date type. Gauss decay origin must be numeric (System.currentTimeMillis()), NOT date-math strings like 'now/d'. Date-math only works on date-typed fields. This is a common OpenSearch footgun.

3.3 6-Leg Retrieval Design — Why 6 Legs?

Each retrieval leg has a different purpose in covering the recall-precision space:

Leg	Query Used	RRF Weight	Purpose
knnOrig	Original query embedding	1.0x	Semantic matches on original intent — anchors recall.
knnRew	Rewritten query embedding	0.7x	Catches docs that use different terminology. Weighted lower because rewrite adds noise risk.
bm25Orig	Original query text, fuzziness=AUTO	1.0x	Exact keyword matches — Jira keys, product names, error codes. BM25 excels where kNN fails.
bm25Rew	Rewritten query text	0.7x	Keyword matches on synonyms/expansions. Same noise discount as knnRew.
custKnn	Original embedding + metadata.customer filter	2.0x	Customer-specific semantic — live ops docs for this customer. 2x weight makes these surface at top.
custBm25	Original text + metadata.customer filter	2.0x	Customer-specific keyword — deployment state, logs. Also 2x.

The base 4 legs have **NO customer/product/env filter**. This is intentional: Jira and Confluence docs don't have metadata.customer set, so filtering by customer on base legs would silently return 0 Jira results. The dedicated customer legs (custKnn, custBm25) handle the strict-scoped live-ops search.

3.4 RRF Fusion Algorithm — Exact Formula

```
RRF_score(chunk) = SUM over lists L:
  (L.listWeight * SourceAuthority(chunk.source)) / (60 + rank_in_L)
```

RRF_K=60 (from the Cormack & Clarke 2009 paper). The constant 60 smooths rank differences — a chunk at rank 1 scores 1/61, rank 10 scores 1/70. Small differences at the top matter more than large differences at the bottom.

Source authority multipliers are hard-coded, not model-determined:

Source	Authority Score	Rationale
LIVE_LOGS	1.0	Real-time evidence — highest ground truth. If logs say X, X is happening.
DEPLOYMENT_STATE	0.9	Nearly as authoritative — deployment state is factual, not interpretation.
CODE	0.8	Source of truth for behaviour, but stale code may exist.
JIRA	0.7	Human-written, may be outdated or inaccurate. But Jira keys are often the best bug references.
CONFLUENCE	0.5	Docs drift fastest — architecture docs written 18 months ago may be wrong.

> *WHY hard-code authority instead of letting the model decide? Because llama3.2:3b is a 3B parameter model — it will hallucinate authority ordering. Hard-coding ensures live logs always outrank docs, regardless of query. This is a deliberate safety decision.*

3.5 Cross-Encoder Reranking — Why It Changes Everything

The first pass (kNN + BM25) uses **bi-encoder** models — query and document are encoded independently. This is fast (pre-computed doc embeddings) but approximate: the model never sees query+doc together.

The cross-encoder (bge-reranker-base) receives **(query, document) pairs** and scores them jointly with full cross-attention. This is 10-100x slower but far more accurate — it can detect whether a document actually answers the query, not just whether it's topically similar.

The pipeline architecture (**retrieve 50** → **rerank top 30** → **take 6**) is the standard industry pattern: cheap approximate retrieval for recall, expensive reranking for precision. The 50→30→6 funnel prevents the reranker from seeing garbage and wastes no reranker capacity on docs that BM25+kNN rejected.

3.6 Source Budget — Preventing Context Monopoly

After reranking, top-6 chunks are selected greedily from the sorted list, subject to per-source caps:

Source	Max Slots	Why This Cap
live_logs	2	Logs can contain thousands of similar lines. 2 representative log entries is enough context.
deployment_state	1	One deployment state doc answers 'what version is running' completely.
jira	1	One matching Jira ticket is highly relevant; more are likely duplicates or related noise.
git	1	One code snippet from the relevant function. More would exceed LLM context budget.
confluence	1	One doc section answers most doc-based questions.

If budget constraints leave fewer than 6 chunks, the algorithm fills remaining slots with the next best candidates regardless of source. This prevents the case where budget is too strict and the LLM gets too little context.

3.7 LLM Generation — Model Choice and Prompt Design

Ollama runs **llama3.2:3b** locally (no data egress). The system prompt includes a customer context header when customer= is set:

```
You are a Searchly intelligence assistant.
Answer using ONLY the context provided.
Cite document titles and Jira issue keys.

CUSTOMER CONTEXT:
  Customer ID : acme-corp
  Environment : prod

CONTEXT:
--- [1] LIVE LOGS | core-service ---
... log lines ...
--- [2] JIRA | AES-891 ---
... ticket content ...

QUESTION: ...
ANSWER:
```

The labels (LIVE LOGS, DEPLOYMENT, JIRA, CODE, DOCS) are emoji-decorated to help the small model understand source type without additional prompting. llama3.2:3b cannot reliably reason about abstract source metadata names but responds well to explicit type labels.

The **120s Ollama timeout** reflects p99 generation time for 1000-token answers on CPU. The **query rewrite call** reuses the same Ollama model — it's a fast 1-sentence generation, typically 600ms.

SECTION 4

Multi-Tenancy Model

Tier-based isolation, IDOR prevention, and the shared/dedicated index decision (L3/L4)

4.1 Three-Tier Isolation Model

Tenant isolation is designed around a cost/isolation tradeoff: most customers share infrastructure; enterprise customers get dedicated everything.

Tier	Tenants	OpenSearch Indices	Kafka Topics	Typical Use
FREE	Shared	documents-shared, chunks-shared	indexing.shared	Trial, small teams. Mandatory tenant_id filter on every query.
STANDARD	Shared	documents-shared, chunks-shared	indexing.shared	Production customers, up to 100K docs. Same shared indices.
PREMIUM	Shared	documents-shared, chunks-shared	indexing.shared	Large production. Shared indices, priority in rate limiter.
ENTERPRISE	Dedicated	documents-{tenantId}, chunks-{tenantId}	indexing.enterprise.{tenantId}	Largest customers. Physical isolation — no shared infra.

4.2 Routing Key Optimization

All OpenSearch queries include **.routing(ctx.tenantId())**. This is not just a filter — it's a routing hint that directs the query to the specific shard(s) containing that tenant's docs, avoiding scatter-gather across all shards. With 3 shards (dev), routing reduces fan-out from 3 to 1 shard per query.

4.3 TenantSecurityFilter — Anti-IDOR

Every request passes through TenantSecurityFilter before reaching any controller. The filter:

- Extracts tenant ID from JWT claim (production) or X-Tenant-Id header (dev/service mode)
- Validates tenant exists in PostgreSQL (Caffeine cache, 5-min TTL)
- Validates user belongs to that tenant (membership check)
- Populates TenantContextHolder — a ThreadLocal cleared after request completes
- Rejects 401 on missing tenant; 403 on tenant mismatch (JWT tenant != path tenant)

The **ThreadLocal pattern** (TenantContextHolder) means tenant context is available anywhere in the call stack without parameter threading. The downside: context must be explicitly propagated to async threads (CompletableFuture captures it via lambda closure at submission time).

SECURITY GAP: acl_users and acl_roles fields are stored in OpenSearch but never enforced at query time. A VIEWER in one sub-team can see documents ACLed to another sub-team within the same tenant. Fixing this requires a bool.should filter on every chunk + document query. This is the #1 security priority before enterprise GA.

4.4 Why Not Separate Databases Per Tenant?

Approach	Pros	Cons	Our Decision
Separate DB per tenant	Complete isolation, independent backup/restore, simple queries	N x DB overhead, N x connection pools, schema migration complexity (N migrations), no cross-tenant analytics	Rejected — 57 customers = 57 Postgres instances, unmanageable
Schema per tenant (same DB)	Database-level isolation, standard pattern	Postgres schema switching overhead, connection pooling complexity	Rejected — doesn't work well with Hibernate/JPA
Row-level isolation (our choice)	Single schema, simple connection pool, standard queries + tenant_id filter	Must never forget the filter, no physical isolation for ENTERPRISE	Accepted — mitigated by TenantSecurityFilter; ENTERPRISE gets dedicated infra
Shared OpenSearch + routing (our choice)	Cost-efficient, single cluster to operate, routing = fast shard targeting	Cross-tenant data in same shard (isolated by filter), potential noisy-neighbour	Accepted — tenant_id filter + routing key is the standard Elasticsearch/OpenSearch pattern

SECTION 5

Storage Layer — Why Each Technology

Five storage systems and the specific reason each was chosen over alternatives (L3/L4)

5.1 PostgreSQL — Source of Truth for Structured Data

PostgreSQL stores tenants, users, roles, document metadata (status, fingerprints), quotas, ACLs, and the knowledge graph (kg_entities, kg_relationships).

Why PostgreSQL over alternatives:

Alternative	Why Not
MySQL	JSONB support is weaker (MySQL JSON type lacks GIN indexing, no jsonb_path_ops). KG properties are JSONB — this matters.
MongoDB	ACID transactions across kg_entities + kg_relationships needed for atomic KG writes. MongoDB multi-document transactions are possible but add complexity.
CockroachDB	Distributed SQL adds latency and operational complexity. Searchly's write patterns are not globally distributed.
Amazon RDS Aurora	Considered for production. Still PostgreSQL-compatible. Valid choice for cloud deployment. Adds vendor lock-in vs self-hosted.

The recursive CTE BFS traversal for KG (`WITH RECURSIVE`) is a key reason for PostgreSQL — it's the cleanest way to implement variable-depth graph traversal without a graph database.

5.2 OpenSearch — Hybrid Search Engine

OpenSearch vs Elasticsearch: OpenSearch is a fork of Elasticsearch 7.10 (before Elastic changed license to SSPL). OpenSearch is Apache 2.0 licensed — no commercial restrictions. For a product that may ship to on-prem enterprise customers, SSPL creates legal risk (SSPL is not OSI-approved open source). OpenSearch is functionally identical for Searchly's use cases: HNSW k-NN, BM25, aggregations, Percolator.

OpenSearch vs Pure Vector Databases (Pinecone, Weaviate, Qdrant):

Pure Vector DB	What's Missing for Our Use Case
Pinecone	No BM25. No full-text search. No highlighting. No aggregations. No document store — Searchly needs all of these. Also: cloud-only, data egress, cost at scale.
Weaviate	BM25 added but not as mature as OpenSearch's Lucene-backed BM25. More complex deployment (schema-first). Lower ecosystem maturity for Java clients.

Pure Vector DB	What's Missing for Our Use Case
Qdrant	No BM25 built-in. Sparse vector support (SPLADE) is an alternative but requires a different embedding model and indexing pipeline.
Milvus	Primarily vector-only. Full-text search is newer addition. Java client less mature. Higher operational complexity (etcd dependency).

KEY INSIGHT: The reason for OpenSearch is not 'it has vectors'. It's that it combines BM25 + kNN + aggregations + document store + highlighting + filtering in one system. Replacing OpenSearch with Pinecone would require a separate Elasticsearch cluster for BM25 — adding complexity we just offloaded.

5.3 Redis — Cache + Rate Limiter

Redis serves two roles: query result cache (60s TTL) and sliding-window rate limit counters.

Cache key design: sha256(tenantId + roles_csv + query + params) — roles are included because a VIEWER and a TENANT_ADMIN may get different results if ACL enforcement is ever added. Not including roles now would be a security bug later.

Why Redis over Caffeine for caching: Caffeine (in-memory) doesn't share state across search-api instances. When horizontally scaled (2 replicas), a cache miss on replica 1 would hit OpenSearch even though replica 2 already cached the result. Redis is the shared L2 cache. Caffeine is used only for tenant config and JWKS (per-process, immutable-ish data).

Resilience gap: Redis failure currently causes 429 errors (rate limiter throws). Fix: catch RedisException, degrade to allow-through. Cache miss on Redis down is acceptable; rejecting all requests is not.

5.4 Kafka — Async Indexing Queue

The search-api returns 202 Accepted immediately after storing metadata in Postgres and bytes in MinIO, then publishes a Kafka message. The indexer consumes asynchronously.

Why Kafka over alternatives:

Alternative	Why Not
RabbitMQ	AMQP with per-message ack. Good for task queues. But Kafka's partitioned log gives replay-ability — if the indexer has a bug, fix it and replay from offset. RabbitMQ messages are gone after ack. Kafka also gives better throughput for bulk indexing bursts.
Direct HTTP (sync indexing)	Indexing a 10MB PDF takes 2-5 seconds (parse + chunk + embed + write). Blocking the upload request for 5s is a terrible UX. Async with 202 is the right pattern.
AWS SQS	Valid for cloud deployment. Adds vendor lock-in. Kafka is self-hosted, works identically on-prem and cloud.

Alternative	Why Not
Redis Streams	Simpler to operate than Kafka. But lacks the partition-based consumer group model and long-term log retention for replay.

Partition design: indexing.shared has 12 partitions (allows 12 concurrent indexer instances). Enterprise tenants get dedicated topics (1 partition default — enterprise volumes are lower but latency requirements higher).

5.5 MinIO — Object Storage for Raw Documents

Raw document bytes (PDFs, Word docs, code files) are stored in MinIO (S3-compatible). The OpenSearch + Postgres layer stores only extracted text and metadata.

Why not store blobs in Postgres: Large BLOBs in Postgres cause table bloat, vacuum overhead, and slow pg_dump. Postgres is optimized for structured queries, not large binary storage. MinIO is purpose-built for object storage at S3 scale.

Why not S3 directly in dev: MinIO allows local development with zero cloud dependency. In production, MinIO can be swapped for S3 by changing the endpoint URL — same SDK.

SECTION 6

Indexing Pipeline

From document upload to searchable chunks — chunking, fingerprinting, embedding versioning (L3/L4)

6.1 Full Indexing Flow

• **Step 1 (search-api):** Receive upload → generate UUID → write metadata to Postgres (status=PENDING) → stream bytes to MinIO → publish Kafka message with doc_id → return 202

• **Step 2 (indexer):** Consume Kafka message → compute SHA-256 fingerprint of content → check Postgres for previous fingerprint → if unchanged, skip chunking+embedding (idempotent replay)

• **Step 3:** Write full document to documents-{index} → chunk text → batch embed (50 chunks/request to embedding-service) → write chunks to chunks-{index}

• **Step 4:** Update Postgres doc status to INDEXED → commit Kafka offset

6.2 Chunking Strategy — Why 2000/200?

Chunks are 2000 characters with 200-character overlap. The overlap ensures a sentence split at a chunk boundary doesn't lose context — the next chunk re-reads the last 200 chars.

Why 2000 chars, not 512 tokens (the common alternative)? Character-based chunking is deterministic and language-agnostic. Token-based chunking requires running the tokenizer for every document, which is slower and tokenizer-dependent. For BGE small (384 dim), the effective input window is 512 tokens (~350 words ~2100 chars). 2000 chars fits this window with a small buffer.

Special case: Files under adr/, decisions/, or architecture/ directories are kept whole if under 12,000 chars. Architecture Decision Records need to be read as a unit — splitting them by character would break the 'Context / Decision / Consequences' narrative.

6.3 Content Fingerprinting — Why SHA-256?

SHA-256 of the extracted text is stored per document. On re-index (e.g., scheduled sync re-fetches the same Jira ticket), if the fingerprint matches, chunking and embedding are skipped.

This is critical for the GitHub connector — it syncs 100+ repos and 2000+ branches. Without fingerprinting, every 4-hour sync would re-embed all unchanged files (millions of chunks).

Why not compare timestamps? Timestamps from external systems (Jira updatedAt, GitHub commit time) are unreliable — clocks skew, bulk imports set wrong times. Content fingerprint is authoritative.

6.4 Embedding Version Tracking

Every chunk stores embedding_version = 'bge-small-en-v1.5-v1'. If the embedding model changes, a migration query can identify all chunks with the old version for re-indexing. The current HNSW index (384

dims) would need rebuilding for a model with different dimensionality.

KNOWN GAP: There is no automated embedding migration pipeline. Upgrading the model currently requires a manual full re-index. For production, versioned index aliases with blue/green promotion would be the correct pattern (index v1 stays live while v2 rebuilds, then alias flips).

6.5 Bulk Indexing — Performance Design

The indexer batches up to 500 documents per OpenSearch /_bulk request. Each /_bulk call is one HTTP round-trip instead of 500. At 50 chunks/doc, a 500-doc batch = 25,000 chunk writes in one call.

Kafka max.poll.records = 10 (application.yml) is an intentional safety limit. Without this, a parallel sync filling Kafka fast could deliver 500 messages at once, triggering 500 concurrent embed calls and 500 database connections — exhausting both the Hikari pool and the embedding service.

SECTION 7

Connectors — Jira, Confluence, GitHub

Delta sync, parallel workers, rate limiting, customer auto-registration (L3)

7.1 Two Sync Tracks

Track	Schedule	Connectors	Why Different Cadence
Track A	Every 60 min	Deployment state (k8s)	Deployment state changes frequently — version upgrades, rollouts happen hourly.
Track B	Every 4 hours	Jira + Confluence + GitHub	Knowledge base changes much slower. Shorter interval would hammer external rate-limited APIs.

7.2 Delta Sync — How It Works

The first run is a full fetch. Subsequent runs add a time filter:

• **Jira:** Appends 'AND updated >= last_completed_at' to JQL. Jira's REST API returns issues sorted by update time — reliable delta.

• **Confluence:** Uses CQL lastModified >= last_completed_at via search API. First run: recursive page walk (depth <= 8). Delta runs: flat CQL result (faster, no recursion needed).

• **GitHub:** Stores per-branch SHA in .sync_state.json. On each run, fetches branch listing (100/page), compares SHA — if unchanged, skips the entire branch tarball fetch. This is the most important optimization: 2000+ branches in DevOps repos, most unchanged.

State file: .sync_state.json on Docker volume. Atomic load-modify-save under a threading.Lock — safe for concurrent workers writing different project keys.

7.3 GitHub — No Git Clone, Streaming Tarballs

GitHub repos are fetched via GET /repos/{org}/{repo}/tarball/{ref}, which returns a gzip-compressed tar stream. The connector uses **tarfile.open(mode='r|gz')** (streaming mode, single pipe) — the tarball is never written to disk.

This is critical for DevOps repos with 2000+ branches. Each branch = one tarball fetch (~5-50MB). Storing them would require 100GB+ of disk. Streaming decompression uses O(chunk_size) memory, then discards the tar.

Why not git clone? git clone copies the entire git object graph (all history). For branches with 5 years of history, this is 500MB per clone. The tarball only contains the working tree at HEAD — exactly what's needed for text extraction.

7.4 Customer Auto-Registration

DevOps repos follow a branch naming convention: {customer-id}-{env} (e.g., acme-corp-nyc-prod, globex-stg). The connector parses each branch:

- _parse_customer_branch() splits on last hyphen: customer_id=acme-corp-nyc, env=prod
- Calls POST /api/v1/customers (idempotent) and POST /api/v1/customers/{id}/environments/{env}
- New customers are auto-registered on their first branch fetch — no manual setup required

Location tokens (nyc, chicago) are intentionally part of the customer ID — the same enterprise customer may have multiple sites and each is a distinct deployment.

SECTION 8

Intelligence Agent — Agentic Loop

Lifecycle stages, Planner bypass, session memory, live k8s credential access, entity resolution (L3/L4)

8.1 Lifecycle Stages — First-Class Concept

Every customer has a lifecycle_stage that determines what data the agent can access:

Stage	Cluster Access	Agent Behaviour	Typical Question
solution	None (no cluster)	Knowledge-only: docs + Jira + code. Answers 'how would this work?'	Will acme-corp's throughput improve with this configuration?
dev	Dev cluster	Full live data from dev cluster. Logs + deployment state.	Why is task X not allocating in dev?
testing	Test cluster	Same as dev but QA context.	We're failing E2E test Y — what does the service log show?
staging	Staging cluste r	Full live data, pre-prod.	Staging shows core service crashing on scenario Z
prod	Prod cluster	Full live data. Prompt emphasises 'be conservative, prefer diagnostic'.	Why is acme-corp's core service down in prod?

8.2 Three-Phase Agentic Loop

The agent runs a **Planner** → **Execution** → **Synthesis** loop:

• **Phase 1 — Planning:** Send the question to Ollama with a system prompt listing available tools. Ask it to output ONLY a JSON array of tool calls. Parse the array with regex (re.search(r'[.*]', raw, re.DOTALL)) — robust to preamble text. Cap at MAX_TOOL_ROUNDS=5.

• **Phase 2 — Execution:** Run all planned tool calls in parallel via asyncio.gather(). Tools are declared as independent (planner outputs a flat list, not a chain). A tool's failure returns {'error': ...} but doesn't stop other tools.

• **Phase 3 — Synthesis:** Append all tool results to the message history (truncated to 8000 chars each). Ask Ollama 'Based on all data gathered above, give your final answer.'

8.3 Knowledge-Only Shortcut — Critical Design Decision

When no cluster is configured (solution phase), the planner is bypassed entirely:

```
if not operational:
    tool_calls_to_run = [{"function": {"name": "search_knowledge",
                                "arguments": {"query": question}}}]
```

Why? llama3.2:3b is a 3B parameter model. Given an open-ended tool selection prompt ('here are 5 tools, decide which to call'), it frequently emits malformed JSON, chooses wrong tools, or hallucinates arguments. When there's only one valid tool (search_knowledge), there's no decision to make — remove the decision from the model entirely.

KEY ARCHITECTURAL PRINCIPLE: Don't trust a small LLM to make decisions it doesn't need to make. The smallest, cheapest model with correct constraints outperforms a larger model with open-ended choices.

8.4 Circular Routing Loop Prevention

Without guards, search_knowledge would create an infinite loop:

```
search_knowledge → GET gateway/api/v1/search
  → SearchService.search() → RagService.answer()
    → IntelligenceAgentClient.chat() → intelligence-agent
      → search_knowledge → ...
```

Two flags break the loop:

• **rag_only=true:** Passed by search_knowledge in tools.py. RagService skips the intelligenceAgent.chat() call when ragOnly=true.

• **hits_only=true:** Also passed by search_knowledge. SearchService skips Ollama generation. The agent does its own synthesis — no need for search-api to also call Ollama.

8.5 Session Memory — Rolling Summary

Conversation context is maintained per session_id (in-memory, known gap — should be Redis):

• 5 verbatim recent turns kept in full

• Older turns compressed into a rolling summary

• Structured memory: {customer, environment, active_issue, investigation_state, known_findings}

• Customer and env resolved in turn 1 are remembered for all subsequent turns — 'what about staging?' works without repeating the customer name

8.6 Live k8s Credential Access — Mode A (Zero Stored Credentials)

The get_logs tool fetches Elasticsearch credentials at runtime:

• kubectl get secret filebeat-credentials -n {namespace} — fetches ES password from k8s Secret

• kubectl exec -n {namespace} {filebeat_pod} -- curl -k -u elastic:{password} https://elasticsearch:9200/... — executes inside the filebeat pod

• No credentials stored in environment variables, no hardcoded passwords

- Credentials are ephemeral — fetched per request, never persisted in memory

Why exec in the filebeat pod? The ES instance is on the internal k8s network, not accessible from outside. The filebeat pod has network access to it. kubectl exec tunnels the curl through the pod's network namespace.

8.7 Entity Resolution — Sliding Token Window

The resolver handles that 'acme' in a user query might be spelled 'Acme Corp', 'ACME London', or 'Acme (London, UK)' in indexed data. The algorithm:

- Slides a 1-4 word window over the full question
- Scores each phrase against the customer registry (max of hint-score and scan-score)
- Matched phrase is learned as an alias for instant future resolution
- Covers any phrasing without depending on NER correctly isolating the customer substring

Why not Named Entity Recognition? llama3.2:3b's NER is unreliable on company names in operational contexts. A deterministic string-matching approach with a learned alias cache is more robust and faster (0ms vs 600ms).

SECTION 9

Knowledge Graph

Schema, API, BFS traversal, and current extraction gaps (L3)

9.1 Why a Knowledge Graph?

Vector search finds semantically similar chunks. But it cannot answer: 'The bug in AES-891 — which PR fixed it and is that PR deployed to acme-corp?' That requires traversal: jira_issue → pull_request → commit → service → deployment.

The KG complements RAG: RAG finds the Jira ticket, KG traversal finds whether the fix is deployed. Together they answer 'is this bug fixed in prod for this customer?' — the #1 question in technical account management.

9.2 Schema (PostgreSQL)

Table	Key Columns	Example Row
kg_entities	entity_type, entity_id, tenant_id, name, properties jsonb	jira_issue | AES-891 | default | 'Redis connection timeout' | {status:'Done', priority:'High'}
kg_relationships	from_type, from_id, relation, to_type, to_id, tenant_id, properties jsonb	jira_issue | AES-891 | fixed_by | pull_request | github.com/go/.../PR/1234 | default

9.3 BFS Traversal — Recursive CTE

```
WITH RECURSIVE graph AS (
  SELECT from_type, from_id, relation, to_type, to_id, 1 AS depth
  FROM kg_relationships
  WHERE from_type = ? AND from_id = ? AND tenant_id = ?
  UNION ALL
  SELECT r.from_type, r.from_id, r.relation, r.to_type, r.to_id, g.depth + 1
  FROM kg_relationships r JOIN graph g ON r.from_type = g.to_type AND r.from_id = g.to_id
  WHERE g.depth < :max_depth AND r.tenant_id = ?
)
SELECT * FROM graph;
```

Max depth is capped at 5 (configurable). Without the depth cap, a circular relationship (e.g., mutual PR-to-ticket links) would cause infinite recursion in the CTE. The JOIN condition ensures traversal follows outbound edges only.

CURRENT STATE: The KG storage layer is complete but extraction is NOT wired. The graph is empty. connector/sync.py does not populate kg_relationships. Priority order: (1) Jira remote links -> jira_issue --[fixed_by]--> pull_request (2) GitHub PR commits (3) File path heuristics ->

service mapping (4) k8s deployment labels -> running service.

SECTION 10

Security & RBAC

JWT, Spring Security, TenantSecurityFilter, GDPR delete (L3/L4)

10.1 RBAC Matrix

Action	VIEWER	EDITOR	TENANT_ADMIN	SERVICE
GET /search	Yes	Yes	Yes	Yes
GET /documents/{id}	Yes	Yes	Yes	Yes
POST /documents	No	Yes	Yes	Yes
DELETE /documents/{id}	No	Yes	Yes	No
Manage tenant config	No	No	Yes	No
POST /kg/entity	No	Yes	Yes	Yes

10.2 JWT Validation Chain

- Gateway validates JWT signature using JWKS endpoint (cached 1h in Caffeine)
- Gateway extracts tenant_id claim, user_id claim, roles claim
- Gateway forwards to downstream services via X-Tenant-Id, X-User-Id, X-Roles headers
- Downstream services trust these headers (no re-validation) — gateway is the trust boundary
- TenantSecurityFilter re-checks tenant exists in DB and user belongs to it

10.3 GDPR Delete — Full Purge Chain

DELETE /documents/{id} performs a full purge:

- OpenSearch: delete from documents-* and chunks-* by doc_id
- PostgreSQL: delete doc metadata record
- MinIO: delete raw file blob
- Redis: invalidate tenant cache (cache.invalidateTenant(tenantId))
- Kafka: publish tombstone message (null value for doc_id key) — signals downstream consumers to delete

KNOWN SECURITY GAP: acl_users and acl_roles stored in OpenSearch but not enforced. All users in a tenant can see all documents regardless of ACL. Fix: add bool.should filter [term(acl_users, userId), term(acl_roles, role_in_roles)] to every chunk + document query.

SECTION 11

Caching Strategy

Three layers with different scopes and TTLs (L3)

Layer	TTL	Scope	Key Design	Invalidation
Redis query results	60s	Cross-instance	sha256(tenantId + roles_csv + query + params)	Any doc mutation calls cache.invalidateTenant()
Caffeine tenant config	5m	Per-instance	tenantId string	On tenant update mutation
Caffeine JWKS	1h	Per-instance	N/A (single entry)	Time-based expiry only

Conversational queries (session_id set) are **never cached** — each turn is unique. Cursor-paginated queries (cursor present) are never cached — each page is unique by sort values.

The 60s TTL is tuned to balance freshness vs. cost. A freshly indexed document might not appear in cached search results for up to 60s. This is acceptable for the use case (near-real-time, not real-time).

SECTION 12

Observability

Designed system — what's wired vs what's not (L3)

Signal	Tool	Status	Notes
Distributed tracing	OpenTelemetry → Jaeger (dev) / Tempo (prod)	Designed	W3C traceparent propagated through Kafka headers. Not fully wired.
Metrics	Micrometer → Prometheus	Partially wired	RED metrics per endpoint per tenant. Instrumentation incomplete.
Logs	Structured JSON (Logback)	Wired	trace_id, span_id, tenant_id, user_id in every log line → Loki / ELK
Retrieval traces	Per-chunk pipeline provenance	Fully wired	RetrievalTrace[30] returned in every SearchResponse. knn_rank, bm25_rank, rrf_score, reranker_score, final_rank, included per chunk.
Eval scheduler	APScheduler nightly 02:00 UTC	Wired	Runs eval_dataset.json against live system. Regression detection at >10% metric drop.
Circuit breakers	Resilience4j	Wired	On embedding-service, reranker, Ollama. 30s default wait in OPEN state.

The retrieval trace is the most important observability feature — it lets you see exactly which chunks from which retrieval legs made it into the final answer, and their scores at each stage. This is essential for debugging 'why did the search return this?' questions.

SECTION 13

Technology Choices vs Alternatives

Every major tech decision with the full reasoning chain (L4)

13.1 LLM: Ollama llama3.2:3b vs Cloud LLMs

Option	Why Considered	Why Not Chosen / Trade-offs
Ollama llama3.2:3b (CHOSEN)	Zero data egress, no cost per token, no latency for network hop, works air-gapped (enterprise on-prem requirement)	Weaker reasoning than GPT-4. 3B param model can't reliably do complex multi-step reasoning. Mitigated by: structured tool design, planner bypass for simple cases, hard-coded authority weights.
OpenAI GPT-4 / GPT-4o	Best reasoning quality, function calling works reliably	Data egress = PII exposure. Enterprise customers cannot allow operational data to leave their network. Cost at scale ($0.01/1K tokens * millions of queries = significant). Latency adds 500ms+ network RTT.
Anthropic Claude 3 Haiku	Fast, cheap, good reasoning	Same data egress concern. No on-prem deployment option.
Ollama mistral:7b or llama3.1:8b	Better reasoning than 3b	8x more RAM required. On a 16GB VM, llama3.2:3b leaves room for all other services. 8B model would require dedicated GPU or much larger VM.

13.2 Embedding Model: BGE bge-small-en-v1.5 vs Alternatives

Model	Dims	Why Considered	Why Not Chosen
BGE bge-small-en-v1.5 (CHOSEN)	384	Excellent MTEB benchmark performance at this size. ~22MB. CPU-deployable. BAAI is well-maintained.	None — best fit for the constraints.
OpenAI text-embedding-3-small	1536	High quality, easy API	Data egress. Cost per embedding (x millions of chunks = significant). Network latency on query path.
all-MiniLM-L6-v2	384	Popular, fast, small	BGE consistently outperforms MiniLM on retrieval benchmarks. Same size, worse results.

Model	Dims	Why Considered	Why Not Chosen
BGE bge-large-en-v1.5	1024	Higher quality embeddings	3x model size (~500MB). Slower inference. OpenSearch HNSW index at 1024 dims = ~4x RAM. Not justified for current query volume.
E5-large or GTE-large	1024	Strong MTEB scores	Same size concerns as bge-large. Also: index rebuild required when switching — high migration cost.

13.3 Search Engine: OpenSearch vs Elasticsearch vs Pure Vector DB

(Covered in depth in Section 5.2. Summary for quick reference:)

Option	Decision
OpenSearch (CHOSEN)	Apache 2.0 license. Identical to Elasticsearch for our features. No commercial restriction for on-prem enterprise delivery.
Elasticsearch	Rejected: SSPL license is not OSI-approved. Legal risk for on-prem enterprise distribution.
Pinecone / Weaviate / Qdrant	Rejected: no BM25 or it's not mature. Searchly needs hybrid — one system for both.
Milvus	Rejected: etcd dependency, higher operational complexity, less mature Java client.

13.4 API Framework: Spring Boot vs Alternatives

Option	Why
Spring Boot 3 (CHOSEN)	JVM virtual threads (Project Loom) handle 100+ concurrent I/O-bound retrieval legs efficiently. Java OpenSearch client is most mature. Resilience4j integrates natively. Spring Security + JWT = battle-tested.
Quarkus	Faster startup, lower memory. But Spring Security and OpenSearch client maturity are not there yet. Ecosystem is smaller.
Micronaut	Same concerns as Quarkus.
FastAPI (Python)	Used for embedding-service and intelligence-agent where Python's ML ecosystem matters. Not ideal for search-api: Python GIL limits multi-threaded I/O parallelism; Java virtual threads are cleaner for 6-leg concurrent retrieval.

Option	Why
Node.js	Good for I/O-bound work, but OpenSearch Java client is significantly more mature than the Node.js client. Java type system also catches schema mismatches at compile time.

13.5 Message Queue: Kafka vs RabbitMQ vs SQS

(Covered in Section 5.4. Summary:)

• **Kafka chosen** for: log retention and replay-ability (indexer bug → fix → replay). Partition-per-tenant Kafka topic design. High-throughput bulk indexing.

• **RabbitMQ rejected**: messages gone after ack, no replay. Good for task queues, bad for event logs.

• **SQS rejected**: valid for cloud, adds vendor lock-in, doesn't work on-prem.

13.6 Session Store: In-Memory vs Redis

Sessions are currently stored in-memory (a known gap). Redis is the right answer for production:

• In-memory: zero operational overhead, works for single-instance dev. Dies on restart — all sessions lost. Doesn't work across multiple search-api replicas.

• Redis: 30-line change to move SessionStore. Sessions survive restart. Shared across replicas. TTL-based cleanup.

• Why not PostgreSQL for sessions?: Too much I/O per turn. Session reads/writes happen on every message. Redis is O(1) hash operations — much faster.

13.7 RRF vs Alternatives for Hybrid Fusion

Fusion Method	Description	Why RRF Chosen
RRF (CHOSEN)	1/(k+rank) summed across lists	Parameter-free, robust to score scale differences between BM25 and kNN (scores are incomparable), proven in TREC and academic benchmarks.
Weighted score combination	alpha * knn_score + (1-alpha) * bm25_score	Requires tuning alpha. BM25 scores (0-20 typical) and kNN scores (0.0-1.0) are in completely different ranges. Normalizing both is error-prone.
Linear combination (normalized)	Normalize both to [0,1] then combine	Normalizing requires knowing max score, which changes per query. Brittle.
Learning-to-rank	Train a ranker on labeled query-doc pairs	Requires labeled training data we don't have. Overkill for current scale.

SECTION 14

Architecture Weaknesses & Known Gaps

An honest engineering assessment — what you'd prioritize fixing and why (L4)

Gap	Severity	Impact	Fix (Effort)
ACL enforcement missing in queries	Critical	Any user in a tenant can read all documents regardless of ACL settings. Sub-tenant data isolation broken.	Add bool.should(acl_users, acl_roles) filter to all chunk + doc queries (2 days)
KG extraction not wired	High	Knowledge graph is empty. Agent cannot answer 'is this bug fixed and deployed?'	Wire Jira remote links first (3 days). Full extraction pipeline (2 weeks).
Session store in-memory	High	Sessions lost on search-api restart. Multiple replicas have split session state.	Move SessionStore to Redis (30-line change, 1 day)
Retrieval legs sequential (was)	High	Was ~300ms wasted sequential time. FIXED in current code — CompletableFuture.allOf()	Already fixed. ~250ms saved.
Recency boost missing from chunk BM25	Medium	Old chunks rank equally with new ones in RAG path. Stale Jira tickets may surface over recent ones.	Add function_score gauss decay to bm25Internal (1 day). Use epoch ms origin — created_at is long, not date.
Redis failure -> 429 all requests	Medium	If Redis goes down, rate limiter throws, all requests fail. Should degrade to allow-through.	Catch RedisException, log warning, return (1 hour)
Eval dataset too small (5 questions)	Medium	Regressions in edge cases won't be detected. Nightly eval is not statistically meaningful.	Need 200+ production-derived cases. Manual labeling or LLM-assisted annotation.
Embedding migration path absent	Medium	Upgrading the embedding model requires full re-index with no zero-downtime path.	Versioned index aliases + blue/green promotion (1 week)
Kafka max.poll.records=10	Low	Intentional safety limit. Could slow burst indexing. Should be dynamically tuned.	Consider raising to 50 with backpressure handling.
Ollama async queue needed	Low	Under heavy load, Ollama requests queue and p99 latency grows unbounded.	Ollama queue + streaming for tail latency (3 days)

SECTION 15

Scalability & Production Path

Index size math, horizontal scaling, and the K8s production target (L4)

15.1 Index Size Math

Scale	Documents	Chunks	Disk (OpenSearch)	RAM (HNSW Index)
Small (current)	~30K docs	~300K chunks	~380 MB	~4 GB
Medium	~300K docs	~3M chunks	~3.8 GB	~16 GB
Large	~3M docs	~30M chunks	~38 GB	~64 GB
XL	~30M docs	~300M chunks	~380 GB	~640 GB

HNSW index for 384-dim float32 vectors: each vector = 1536 bytes. 300K vectors = ~460MB in RAM just for HNSW. Plus Lucene segment RAM. At 30M chunks a single OpenSearch node won't hold the index in RAM — sharding across a cluster required.

15.2 Horizontal Scaling Points

• **search-api:** Stateless (TenantContextHolder is ThreadLocal, not process-local). Scale horizontally behind a load balancer. Move session store to Redis first.

• **embedding-service:** Stateless (model loaded on startup, no write state). Scale with multiple replicas behind a load balancer.

• **intelligence-agent:** Stateless (session state in-memory is the gap). Same fix as search-api.

• **indexer:** Scale by adding Kafka partition consumers. One JVM per partition is the correct model.

• **OpenSearch:** Add data nodes. Increase shard count (requires index rebuild). HNSW scales horizontally — each shard holds a fraction of vectors.

• **Kafka:** Add brokers. Increase partitions on indexing.shared (requires consumer group restart).

15.3 Production K8s Target

• **Blue-green deployment** via two Deployments + Service selector flip. Zero-downtime rollouts.

• **Argo Rollouts** for canary: route 5% traffic to new version, watch error rate, promote or roll back.

• **Flyway** DB migrations: expand-then-contract pattern — never drop columns in the same release that stops writing them.

• **Build tool: Maven** (explicit preference — Gradle not used in this project).

• **Push to git** → **pull on VM** → **docker compose up -d --build** is the current dev deploy. Never scp files directly.

SECTION 16

Interview Talking Points & Likely Questions

What to lead with, what to prepare for, and how to frame the tradeoffs (L4)

16.1 The Opening 2-Minute Pitch (System Design)

'Searchly is a multi-tenant RAG + agentic platform. The core insight is that enterprise operational data is fragmented: Jira tickets, Confluence docs, GitHub code, and live k8s logs are all in silos. We built a 6-leg hybrid retrieval pipeline — BM25 for exact matches, kNN for semantic, RRF fusion, then cross-encoder reranking — over OpenSearch. The intelligence agent wraps this with a Planner-Execution-Synthesis agentic loop that can pull live pod logs and deployment state alongside indexed knowledge. The hard problems were: multi-tenant isolation without N database instances, preventing circular agent routing, and making a 3B parameter model reliably emit structured tool plans by removing choices it didn't need to make.'

16.2 Likely Deep-Dive Questions and What to Emphasise

Q: How does RRF work and why not weighted score combination?

Explain the 1/(k+rank) formula. Key point: BM25 and kNN scores are in completely different numeric ranges — you can't add them directly. RRF uses only rank position, which is universal. The 60 constant smooths differences at the top of the list. Authority weighting happens inside the RRF multiplier so source quality is captured without score normalisation.

Q: Why llama3.2:3b instead of GPT-4?

Data egress is the hard constraint — enterprise operational data cannot leave the customer's network. 3B runs on CPU with 4GB RAM. The model limitation is mitigated by: removing decisions it doesn't need to make (knowledge-only shortcut), hard-coding authority weights so the model doesn't rank sources, and structured prompts.

Q: How do you prevent cross-tenant data leakage?

Three layers: (1) TenantSecurityFilter validates JWT claim matches path tenant and user membership before any query runs. (2) Every OpenSearch query includes .routing(tenantId) + term filter on tenant_id. (3) ENTERPRISE tier gets physically separate indices. The known gap is sub-tenant ACL enforcement — acl_users/acl_roles stored but not yet enforced in queries.

Q: How does the agent prevent infinite loops?

Two orthogonal flags: rag_only=true (search-api skips intelligence-agent.chat() call) and hits_only=true (search-api skips Ollama, returns raw chunks). search_knowledge passes both. The circular path is search_knowledge -> gateway -> search-api -> agent -> search_knowledge. rag_only breaks the middle -> agent step.

Q: How does delta sync work for Jira?

JQL filter: 'AND updated >= last_completed_at' added from the second run onwards. State stored per-project in .sync_state.json. First run fetches all, stamps completed_at. Second run only fetches changed issues. SHA-256 fingerprint check means re-fetched but unchanged content skips re-embedding.

Q: What would you change if you had 3 months?

Priority order: (1) ACL enforcement — security first. (2) Wire KG extraction — unlocks jira->PR->deployment traversal. (3) Redis session store — enables horizontal scale. (4) Recency boost on chunk BM25 — better relevance. (5) Embedding migration pipeline — enables model upgrades without downtime.

Q: How would you scale this to 10M chunks?

OpenSearch multi-node cluster (3-5 data nodes, 1 master). Increase shard count (requires index rebuild — plan with versioned aliases). HNSW at 10M x 384 dims needs ~16GB RAM in the index — spread across shards. Kafka: increase indexing.shared to 24 partitions, 24 indexer replicas. search-api: horizontal scale is already designed (stateless after Redis session fix). Add a query analysis layer to detect expensive queries and circuit-break early.

16.3 The 3 'Why Not' Answers You Must Know Cold

• **Why not Elasticsearch?** — SSPL license. Not OSI-approved open source. Cannot ship to enterprise on-prem without potential legal exposure. OpenSearch is the Apache 2.0 fork — functionally identical for our use case.

• **Why not a pure vector database (Pinecone/Weaviate)?** — We need BM25 for exact keyword matching (Jira keys, error codes, product names). Pure vector DBs don't have this or it's immature. Hybrid search in one system is the right architecture.

• **Why not LangChain?** — LangChain abstracts retrieval logic in ways that made it harder to implement the specific RRF formula, the custom authority weights, the source budget, and the planner bypass. For production RAG with specific latency and correctness requirements, owning the pipeline gives full observability and control. LangChain is great for prototypes; hand-rolled pipelines are better for production.

16.4 Numbers to Know by Heart

Metric	Value	Context
Retrieval latency (wall-clock)	~50ms	6 parallel legs via CompletableFuture.allOf() on virtual thread pool
Embedding dim	384	BGE bge-small-en-v1.5. 1536 bytes per vector.
RRF K constant	60	From Cormack & Clarke 2009 paper. Standard value.
Rerank candidates	Top 30	From RRF result. Cross-encoder scores all 30, selects 6.
Context chunks	6	Source-budget-balanced. Fits llama3.2:3b context window.
Query rewrite time	~600ms	Ollama llama3.2:3b, 1 sentence output. Skipped for bare Jira keys.
Ollama generation time	~4s	p50. p99 can be 10s+ under concurrent load.
Cache TTL	60s	Redis. sha256 key. Conversational queries never cached.
Chunk size	2000 chars / 200 overlap	~1 page of text. ADR files kept whole if < 12K chars.
Max retrieval candidates per leg	50	CANDIDATE_K. Top 30 sent to reranker.
Source authority: LIVE_LOGS	1.0	Highest. Real-time ground truth.
Source authority: CONFLUENCE	0.5	Lowest. Docs drift fastest.
BM25 k1	1.2	OpenSearch default. Term frequency saturation.
BM25 b	0.75	OpenSearch default. Length normalization.
Max agentic tool rounds	5	MAX_TOOL_ROUNDS in agent.py
Jira rate limit	7 req/s	_ATLASSIAN_RL shared across Jira + Confluence workers
GitHub rate limit	5 req/s	_GITHUB_RL. Default 1 worker in prod to bound peak memory
Kafka indexing.shared partitions	12	Allows 12 concurrent indexer instances
Hikari pool size	10	Default. Exhausted when >10 concurrent sync runs hit DB

Metric	Value	Context
Index size estimate (300K chunks)	~380MB disk / ~4GB RAM	HNSW 384-dim. Scales linearly.

Good luck with the interview.

This document was generated from the live Searchly codebase and reflects the actual implementation.

Version: June 2026 · Searchly Platform